

```
❶ if alien.rect.bottom >= self.settings.screen_height:
    # Treat this the same as if the ship got hit.
    self._ship_hit()
    break
```

The method `_check.aliens.bottom()` checks whether any aliens have reached the bottom of the screen. An alien reaches the bottom when its `rect.bottom` value is greater than or equal to the screen's height ❶. If an alien reaches the bottom, we call `_ship_hit()`. If one alien hits the bottom, there's no need to check the rest, so we break out of the loop after calling `_ship_hit()`.

We'll call this method from `_update.aliens()`:

```
alien_invasion.py def _update.aliens(self):
    --snip--
    # Look for alien-ship collisions.
    if pygame.sprite.spritecollideany(self.ship, self.aliens):
        self._ship_hit()

    # Look for aliens hitting the bottom of the screen.
    self._check.aliens.bottom()
```

We call `_check.aliens.bottom()` after updating the positions of all the aliens and after looking for alien-ship collisions. Now a new fleet will appear every time the ship is hit by an alien or an alien reaches the bottom of the screen.

Game Over!

Alien Invasion feels more complete now, but the game never ends. The value of `ships_left` just grows increasingly negative. Let's add a `game_active` flag, so we can end the game when the player runs out of ships. We'll set this flag at the end of the `__init__()` method in `AlienInvasion`:

```
alien_invasion.py def __init__(self):
    --snip--
    # Start Alien Invasion in an active state.
    self.game_active = True
```

Now we add code to `_ship_hit()` that sets `game_active` to `False` when the player has used up all their ships:

```
alien_invasion.py def _ship_hit(self):
    """Respond to ship being hit by alien."""
    if self.stats.ships_left > 0:
        # Decrement ships_left.
        self.stats.ships_left -= 1
        --snip--
        # Pause.
        sleep(0.5)
    else:
        self.game_active = False
```
